Introdução ao Lean 4 e à Verificação Formal de Provas

Dia 1: Fundamentos, Lógica e Sintaxe

Equipe do projeto de Inovação em Lean 4
Departamento de Matemática e Matemática Aplicada
Universidade Federal de Lavras

Minicurso — 2026

Roteiro do Dia 1

1. Como saber se uma demonstração é correta?
2. O que é o Lean
3. O que o Lean tem a ver com a IA?
4. O que o Lean **não** faz: SAT/SMT e o espectro das ferramentas
5. O que o Lean **faz**: tipos dependentes, Curry–Howard e o kernel
6. IA, Lean e pesquisa matemática
7. Sintaxe do Lean 4: termos e táticas
8. Ambiente de trabalho: LSP e estado da prova
9. Ecossistema: Mathlib e onde encontrar o Lean

Link

https://github.com/rfranquiz/Mini-curso_Lean_UFLA_2026

Como saber se uma demonstração é correta?

A pergunta

Existe uma forma de **checar mecanicamente** se uma prova está correta?

Resposta 1: Revisão por pares!

Processo no qual um trabalho ou sistema é avaliado e validado por pessoas que possuem o mesmo nível de conhecimento, especialidade ou status que o autor original.

Como saber se uma demonstração é correta?

A pergunta

Existe uma forma de **checar mecanicamente** se uma prova está correta?

Resposta 1: Revisão por pares!

Processo no qual um trabalho ou sistema é avaliado e validado por pessoas que possuem o mesmo nível de conhecimento, especialidade ou status que o autor original.

Resposta 2: Assistentes de prova!

Ferramentas de programação projetadas para o computador verificar, passo a passo, se uma demonstração matemática ou lógica é 100% correta *em cada passo lógico*.

Como saber se uma demonstração é correta?

A pergunta

Existe uma forma de **checar mecanicamente** se uma prova está correta?

Resposta 1: Revisão por pares!

Processo no qual um trabalho ou sistema é avaliado e validado por pessoas que possuem o mesmo nível de conhecimento, especialidade ou status que o autor original.

Resposta 2: Assistentes de prova!

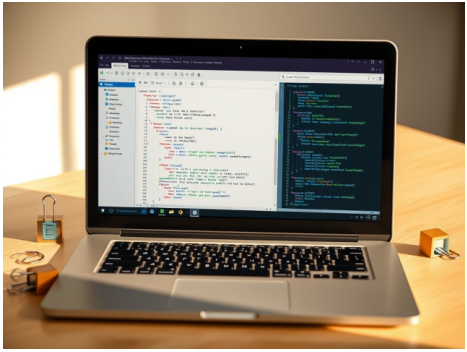
Ferramentas de programação projetadas para o computador verificar, passo a passo, se uma demonstração matemática ou lógica é 100% correta *em cada passo lógico*.

Objetivo!

Garantir a validade, a completude e a ausência de lacunas lógicas no trabalho!

O que é o Lean?

O Lean 4 é uma linguagem de programação que permite traduzir matemática e lógica abstrata em código verificável de forma que o computador possa checar se um raciocínio ou estrutura lógica está 100% correto por meio de uma verificação matemática rigorosa, sem margem para ambiguidades.



LEAN
THEOREM PROVER

O que é o Lean?

- É um projeto de código aberto hospedado no GitHub.
- Foi primordialmente desenvolvido pelo brasileiro Leonardo de Moura, doutor em ciências da computação pela PUC-Rio.
  Leonardo desenvolveu este projeto enquanto trabalhava na Microsoft Research (hoje em dia na Amazon Web Services).
- Em 2013 foram lançadas as versões iniciais conhecidas como Lean 1 e 2. Hoje se encontra na versão 4.33.1 (última atualização 21 de Augosto de 2026).
- O paradigma principal do Lean 4 é a programação funcional. Essencialmente, no Lean a computação é pensada como a avaliação de funções matemáticas.
- Atualmente, o desenvolvimento é apoiado pela organização sem fins lucrativos Lean Focused Research Organization (FRO).

O que Lean tem a ver com a IA?

Mito

Se a IA escreveu a demonstração "convincente", então o resultado está provado!

O que Lean tem a ver com a IA?

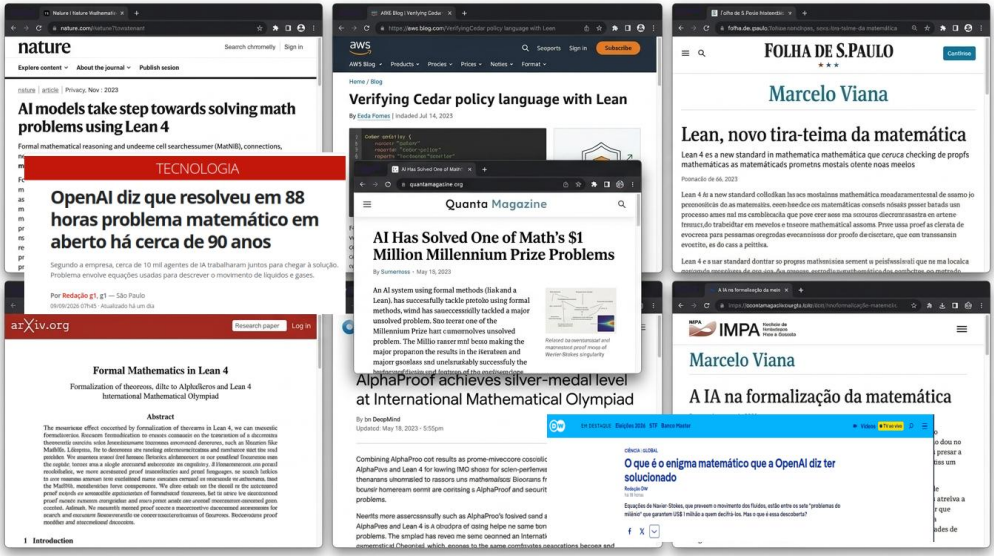
Mito

Se a IA escreveu a demonstração "convincente", então o resultado está provado!

Realidade

Os modelos LLM trabalham com probabilidade, logo podem alucinar. Estes modelos conseguem apontar possíveis falhas em demonstrações, mas não conseguem responder se uma demonstração possui efetivamente um bug.

O que Lean tem a ver com a IA?

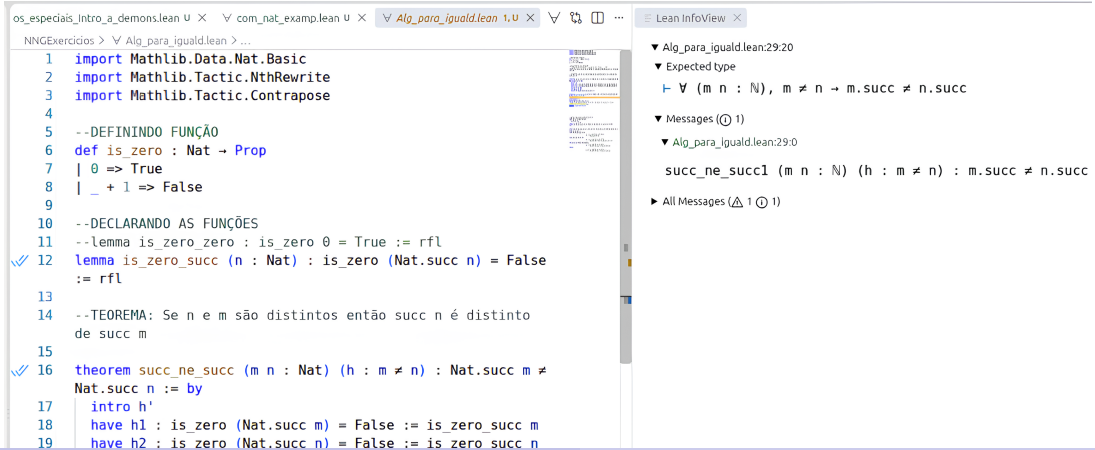


O que Lean tem a ver com a IA?

Os Grandes Modelos de Linguagem (LLMs), modelos de IA baseados em linguagem, trabalham com probabilidades. Ao tentar resolver um problema complexo de matemática ou gerar um código de programação, apesar de parecer convincente, a IA pode cometer erros sutis (alucinar).

O que o Lean tem a ver com a IA?

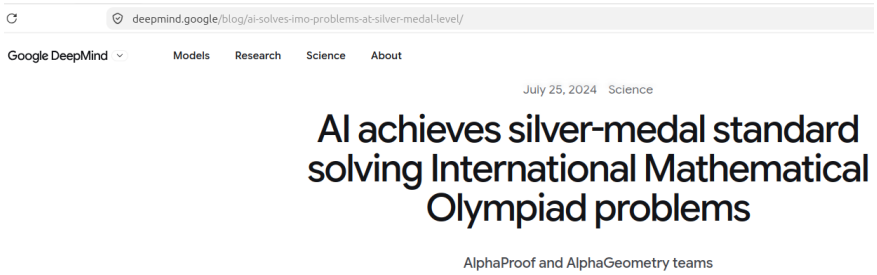
Quando a IA escreve seu raciocínio na linguagem Lean 4, o código passa pelo compilador do Lean, que dá um veredito binário absoluto: ou a lógica é matematicamente válida, ou está errada.



O que é o Lean e o que tem a ver com a IA?

Provas formais em Lean são dados de treinamento excepcionais para modelos de IA, pois possuem verificação automática de correção. Diferentemente das provas matemáticas comuns, nos quais o modelo não dispõe de um mecanismo para determinar se o conhecimento está correto, cada passo de uma prova no Lean é verificado mecanicamente, garantindo sua validade.

IA, Lean e Matemática Olímpica (2024)



AlphaProof — DeepMind (2024)

- Resolveu 4 dos 6 problemas da IMO 2024
- Os enunciados foram traduzidos por humanos para Lean; AlphaProof buscou provas automaticamente
- O Lean serve de **juiz formal**: a IA propõe uma prova, o Lean aceita ou rejeita — sem ambiguidade.

O que o Lean **NÃO** faz!

Mito

"O computador verifica uma prova testando casos, ou decidindo sozinho se o enunciado é verdadeiro."

O que o Lean **NÃO** faz!

Mito

"O computador verifica uma prova testando casos, ou decidindo sozinho se o enunciado é verdadeiro."

Realidade

O Lean não avalia fórmulas nem procura modelos: ele confere um *objeto de prova* construído por alguém (uma pessoa ou uma IA).

O que o Lean **NÃO** faz!

O Lean **não** verifica se uma afirmação é tautologia (verdadeira para todas as atribuições) nem se é satisfazível (verdadeira para pelo menos uma atribuição).

Por quê?

O que o Lean **NÃO** faz!

O Lean **não** verifica se uma afirmação é tautologia (verdadeira para todas as atribuições) nem se é satisfazível (verdadeira para pelo menos uma atribuição).

Por quê?porque o Lean não opera com atribuições de valores. Não há uma estrutura $\mathcal{M}$ onde variáveis recebem valores e fórmulas são avaliadas.

O que o Lean **NÃO** faz!

O Lean **não** verifica afirmações por métodos semânticos de lógica de primeira ordem (verdade em todos os modelos), **nem** por métodos sintáticos de lógica de primeira ordem (usando sistemas de dedução e prova), **nem** verifica satisfazibilidade SMT[a] (verdade em pelo menos um modelo com teorias).

[a] SMT são as siglas para Satisfazibilidade Módulo Teorias

Por quê?

O que o Lean **NÃO** faz!

O Lean **não** verifica afirmações por métodos semânticos de lógica de primeira ordem (verdade em todos os modelos), **nem** por métodos sintáticos de lógica de primeira ordem (usando sistemas de dedução e prova), **nem** verifica satisfazibilidade SMT[a] (verdade em pelo menos um modelo com teorias).

[a] SMT são as siglas para Satisfazibilidade Módulo Teorias

Por quê? Porque as proposições do Lean não são fórmulas de lógica de primeira ordem.

SAT/SMT e Lean

- SAT/SMT você dá uma proposição, a máquina verifica automaticamente se ela pode ser verdadeira — sem nenhuma ajuda humana.
- Lean verifica se um *objeto de prova construído pelo usuário* tem o tipo correto — problema fundamentalmente diferente.

O que o Lean faz!

Mito

"Verificação formal é uma caixa-preta: não dá para saber o que a máquina checou."

Realidade

Um programa pequeno e isolado, o *kernel*, checa uma única coisa: se o termo t tem o tipo P. Provar é construir; verificar é checar o tipo.

O que o Lean faz!

O **Lean** verifica afirmações usando Teoria de Tipos Dependentes.

- **Teoria de Tipos** estuda sistemas formais que classificam expressões ou objetos matemáticos através de seus tipos.

Um *Tipo* é uma noção primitiva que pode ser entendida como uma estrutura (uma coleção de objetos da mesma natureza) munida de regras e operações que se aplicam àqueles elementos.

- **Teoria de Tipos Dependentes** é uma extensão da Teoria de tipos onde os tipos podem depender de elementos (termos) de outros tipos. Diremos que um tipo é dito "dependente" se sua definição depende de um valor.

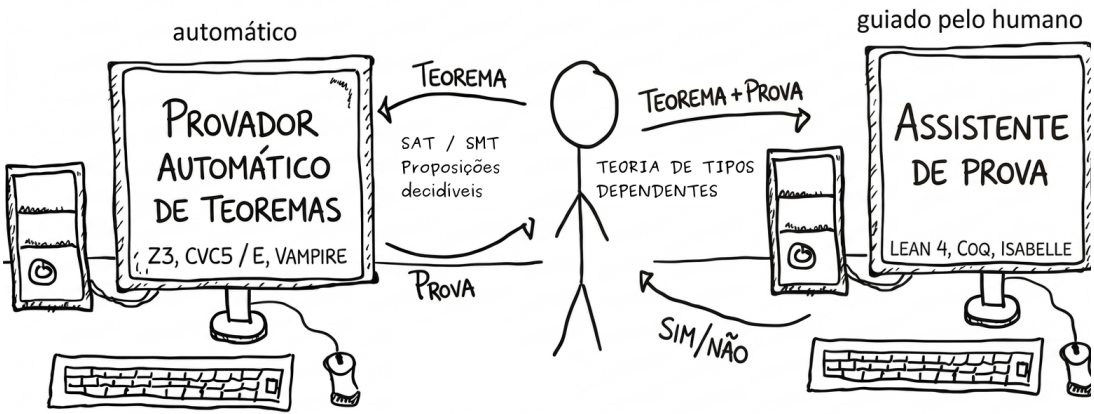
O que o Lean faz!

Exemplo:

Na teoria de tipos, `Lista de Naturais` é um tipo. (Como expressão matemática pode ser visto como um monoide livre gerada por $\mathbb{N}$).

Na teoria de tipos dependentes, um tipo `Vector(n)` seria uma coleção formada por "uma lista de naturais que possui exatamente n elementos". Neste caso, `Vector(n)` é um tipo depende de do parâmetro n.

O espectro das ferramentas de verificação



Dois universos no Lean: Prop e Type

`Prop` — Universo das Proposições

- Elementos são **provas** de proposições lógicas
- Exemplos: $2 + 2 = 4$, $\forall n,\ n + 0 = n$
- Não geram código executável

`Type` — Universo dos Tipos

- Elementos são **dados computacionais**
- Exemplos: `Nat`, `String`, `List Nat`
- Valores são distinguíveis

A correspondência de Curry–Howard

Matemática $\longleftrightarrow$	**Programação (tipos)**
Proposição P	Tipo P
Prova de P	Termo $t : P$
$P \Rightarrow Q$	Função $P \to Q$
$P \wedge Q$	Par ordenado $P \times Q$
$P \vee Q$	União disjunta $P \oplus Q$
$\forall x, P(x)$	Tipo dependente $\prod_{x:A} P(x)$
$\exists x, P(x)$	Par dependente $\sum_{x:A} P(x)$

O que o Lean faz quando você escreve uma prova?

1. Você escreve um **termo de prova** t e declara `theorem meu_lema : P := t`
2. O **kernel de verificação** do Lean checa se $t : P$ é válido segundo as regras da teoria de tipos dependentes
3. Se a verificação passa, a prova está **100% correta** — não existe heurística
4. Se falha, o Lean reporta um erro de tipo (como um compilador)

Ideia-chave

Demonstrar uma proposição = **construir** um termo do tipo correspondente.

O Lean verifica que o termo está bem tipado — isso *é* a verificação da prova.

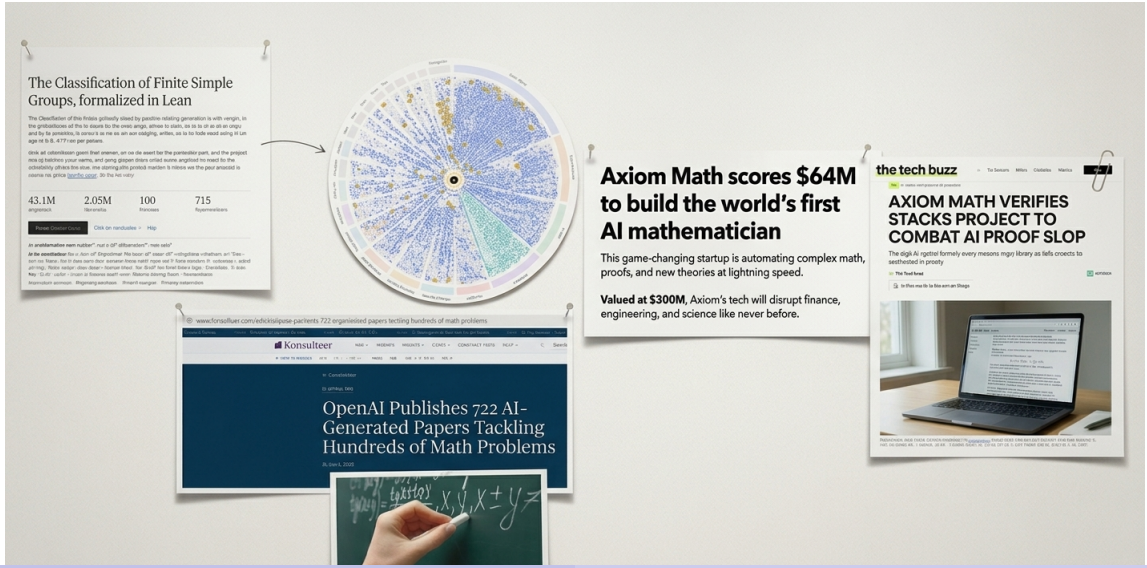
O que o Lean faz quando você escreve uma prova? (O kernel do Lean)

O **kernel** do Lean é um programa pequeno, simples, isolado e extremamente protegido, cuja única função na vida é fazer uma coisa:

verificar se um termo possui um determinado tipo é um programa, ou seja, dado um termo `t` e um tipo `P`, verifica se `t : P` é válido segundo as regras da teoria de tipos dependentes.

Visto de outra forma, o **kernel** (ou núcleo) do Lean 4 é o coração lógico do sistema.

IA, Lean e Pesquisa Matemática



OpenAI- ChatGPT

Em outubro de 2026, a OpenAI publicou de forma massiva 722 artigos acadêmicos gerados por um modelo de IA interno e não lançado, alegando ter resolvido ou avançado em cerca de 372 problemas em 17 áreas da matemática pura.

Em vez de confiar apenas no texto gerado, a IA traduziu as demonstrações para a linguagem de programação matemática **Lean**.

Axiom

Axiom Math- AxiomProver

A Axiom Math é uma startup fundada em 2025 de inteligência artificial que desenvolve o AxiomProver focado no raciocínio matemático e na eliminação de "alucinações".

Até o momento, Axiom tem conseguido US$ 64 milhões em investimento e a empresa foi avaliada em US$ 1,6 bilhão.

Em 2026 Axiom Math conseguiu:

- formalizar em Lean as demonstrações do Stacks Project (o maior livro colaborativo cobrindo áreas complexas de geometria algébrica).
- formalizar em Lean a demonstração da classificação de grupos simples que possui mais de 10.000 páginas e centenas de artigos.

Programas são expressões matemáticas

O λ-cálculo é a base

No Lean, funções são escritas como no λ-cálculo de Church:

$$\lambda x.\, x + 1 \qquad \text{em Lean:} \qquad \texttt{fun x => x + 1}$$

Aplicação: $(\lambda x.\, x + 1)\, 3 = 4$ em Lean: `(fun x => x + 1) 3`

Matemática

$$f : \mathbb{N} \to \mathbb{N}$$
$$f(n) = n + 1$$

Lean 4

```
def f : Nat -> Nat :=
  fun n => n + 1
```

Programas funcionais no Lean *não têm efeitos colaterais*: toda expressão tem um valor determinístico. Isso é exatamente como funções matemáticas.

O protocolo LSP e o ambiente de trabalho

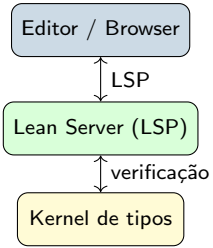
Language Server Protocol (LSP)

- O Lean implementa uma versão estendida do chamado *Language Server Protocol*, que fornece *feedback* enquanto o usuário digita.

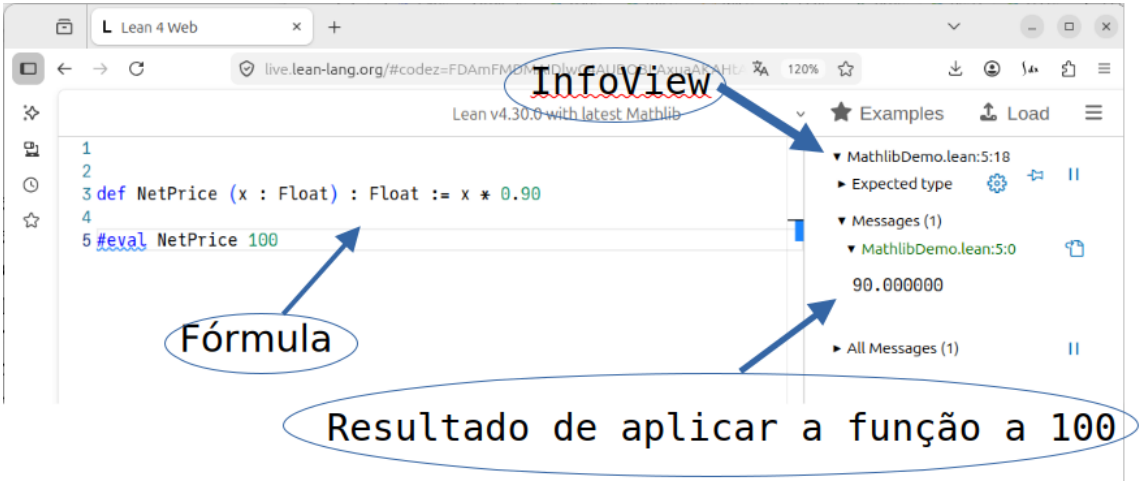
  Isso permite o **painel de estado da prova**: você vê o estado dos objetivos em tempo real.
- Este protocolo trabalha nos editores VS Code ou Emacs

No minicurso: Lean Web

`live.lean-lang.org`
Sem instalação. LSP roda no servidor deles.



O protocolo LSP e o ambiente de trabalho



Preparação para a sessão ao vivo

Ambiente

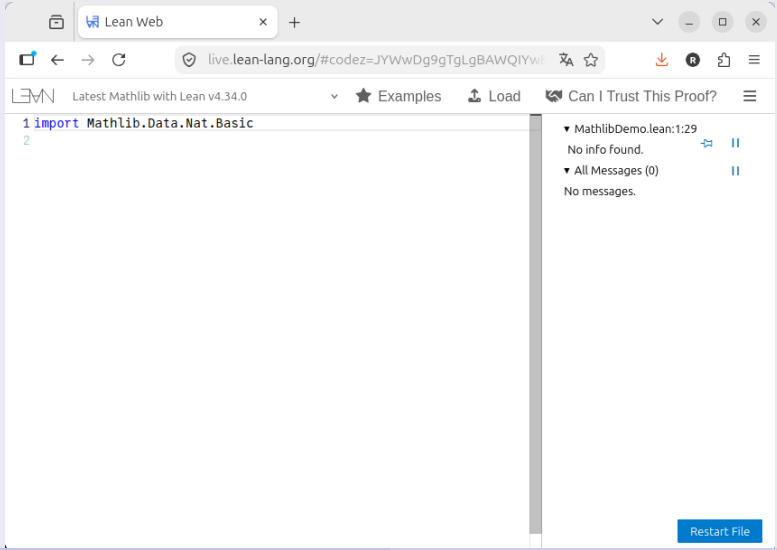
`live.lean-lang.org`

- Sem instalação. Abra no navegador agora.
- O Lean 4 com Mathlib roda no servidor.
- O painel direito (*Infoview*) mostra o estado das provas.

Cole este cabeçalho para começar

```
import Mathlib.Data.Nat.Basic
```

Preparação para a sessão ao vivo



Preparação para a sessão ao vivo

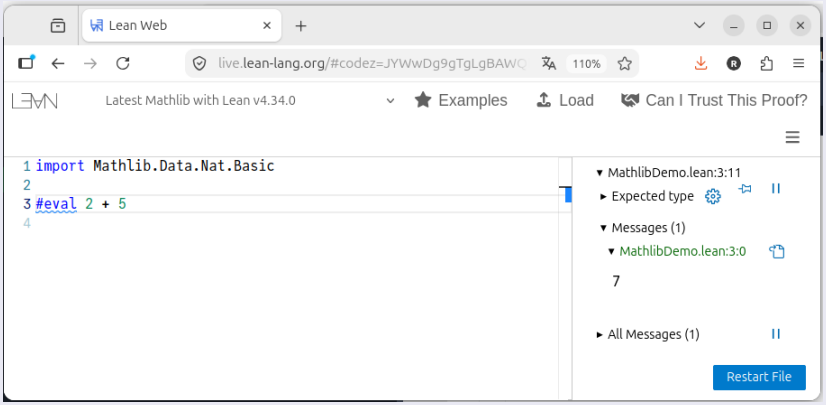
Digite no Lean Web

```
#eval 2 + 5
```

O que deve aparecer

No painel da direita (*Infoview*), em Messages: 7

Preparação para a sessão ao vivo



O que o Lean faz quando você escreve uma prova? (Exemplo)

Imagine que você escreveu o seguinte no Lean:

```
theorem meu_teorema :  2 + 2 = 4 := by rfl
```

- `meu_teorema` é o nome da prova.
- `2 + 2 = 4` é o Teorema (a proposição).
- `rfl` é a prova do teorema

- O tipo de `meu_teorema` é `2 + 2 = 4`.
- O tipo de `2 + 2 = 4` é `Prop`

O que o Lean faz quando você escreve uma prova? (Exemplo)

```
theorem meu_teorema :  2 + 2 = 4 := by rfl
```

Para o kernel do Lean, o processo é mais rígido:

- Ele olha para a sua afirmação (`2 + 2 = 4`) e verifica se ela é um tipo válido em Prop.
- Ele olha para a sua prova (`rfl`) e verifica se ela é um habitante válido daquele tipo (`2 + 2 = 4`).

O que o Lean faz quando você escreve uma prova? (Exemplo)

```
theorem meu_teorema :  2 + 2 = 4 := by rfl
```

Para o kernel do Lean, o processo é mais rígido:

- Ele olha para a sua afirmação (`2 + 2 = 4`) e verifica se ela é um tipo válido em Prop.
- Ele olha para a sua prova (`rfl`) e verifica se ela é um habitante válido daquele tipo (`2 + 2 = 4`).

O termo `rfl` é uma prova de que `a = a`. Nativamente, `rfl` sabe provar que $4 = 4$.

O kernel simplifica e computa os dois lados da igualdade até onde for possível.

Ele calcula $2 + 2$ e reduz para 4.

Para o kernel do Lean, as proposições `2 + 2 = 4` e `4 = 4` são por definição iguais. Ou seja, `rfl` é um habitante válido do tipo `2 + 2 = 4`.

Declarações fundamentais do Lean 4

```
-- def: define um objeto (dado ou funcao)
def meuNumero : Nat := 42
def dobro (n : Nat) : Nat := 2 * n

-- #check: pergunta o tipo de uma expressao
 #check dobro           -- dobro : Nat -> Nat
 #check dobro 5         -- dobro 5 : Nat
 #eval  dobro 5         -- 10

-- theorem / lemma: define uma prova
theorem dobro_par (n : Nat) : dobro n % 2 = 0 := by
  simp [dobro]
```

- `#check` e `#eval` são comandos interativos — o LSP responde em tempo real
- `by` abre o *modo de táticas*: você descreve a prova em passos
- Sem `by`, você escreve o *termo de prova* diretamente

Duas formas de provar: termos vs. táticas

Modo Termo (Prova Explícita)

Você escreve a função ou prova inteira de uma vez só, fornecendo a expressão exata.

```
-- Modus ponens: dado p e p -> q, devolve q
theorem mp (p q : Prop) (hp : p) (hpq : p -> q) : q := hpq hp
```

Mais conciso. Exige ver o tipo do objeto de prova.

Modo Tática (Estratégia Passo a Passo)

Lean entra em um estado interativo que mostra o que falta provar. Você aplica táticas que transformam esse objetivo até não restar nada a provar.

```
theorem mp (p q : Prop) (hp : p) (hpq : p -> q) : q := by
  apply hpq     -- reduz objetivo de q para p
  exact hp      -- fecha o objetivo p
```

Mais próximo da escrita matemática usual.

Táticas principais que usaremos

Tática	**O que faz**	**Análogo matemático**
`rfl`	Prova igualdade por computação	"é imediato"
`rw [h]`	Reescreve usando hipótese h	substituição
`simp`	Simplifica com lemas automáticos	simplificação algébrica
`ring`	Prova identidades em anéis comutativos	álgebra
`intro`	Introduz hipótese de um $\forall$ ou de uma implicação	"seja x"
`apply`	Aplica um lema ao objetivo atual	modus ponens
`exact`	Fornece o termo de prova diretamente	q.e.d.
`induction`	Abre casos base e indutivo	indução

Importante: **táticas** são diferentes de **nomes de lemas**.

Estado da prova: o painel mais importante

Ao escrever `by`, o Lean mostra o **estado dos objetivos**:

```
theorem ex (n : Nat) :
    n + 0 = n := by
  -- cursor aqui
```

Painel lateral (Infoview)

```
n : Nat
⊢ n + 0 = n
```

- Acima de $\vdash$: **contexto** (hipóteses disponíveis)
- Após $\vdash$: **objetivo** (o que falta provar)

Leitura como matemático

$$\underbrace{n : \mathbb{N}}_{\text{hipóteses}} \vdash \underbrace{n + 0 = n}_{\text{tese}}$$

Cada tática *transforma* esse estado até que o objetivo seja vazio.

Dica para os dois dias

Dica!

- Coloque o cursor *dentro* de um bloco `by` para ver o estado da prova.
- Se aparecer `No goals` em verde: a prova está completa!

O que é o Mathlib?

Mathlib4

- Biblioteca matemática unificada da comunidade Lean 4
- Possui mais de 220 000 definições e teoremas formalmente verificados
- Cobre: álgebra (grupos, anéis, módulos, corpos), topologia, análise, teoria dos números, geometria, combinatória, ...
- Mantida por mais de 300 colaboradores; pull requests revisados diariamente

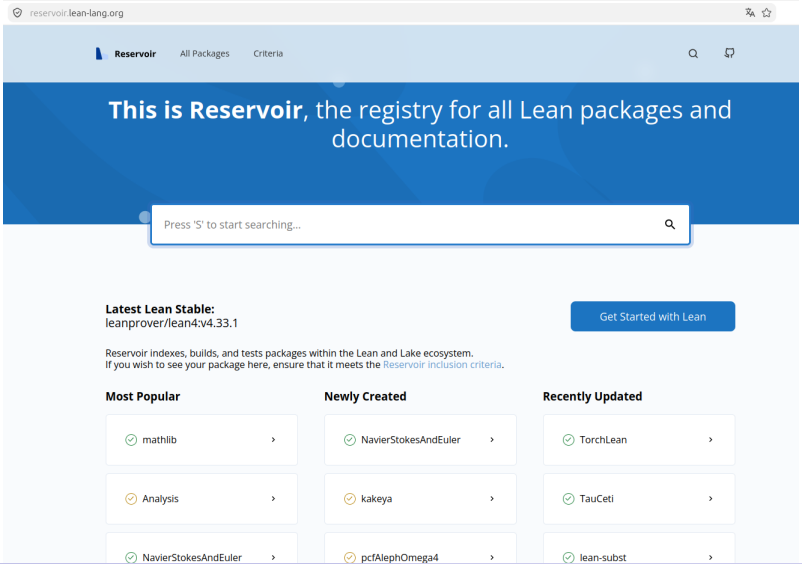
Para usar no seu arquivo

```
import Mathlib.Data.Nat.Basic
```

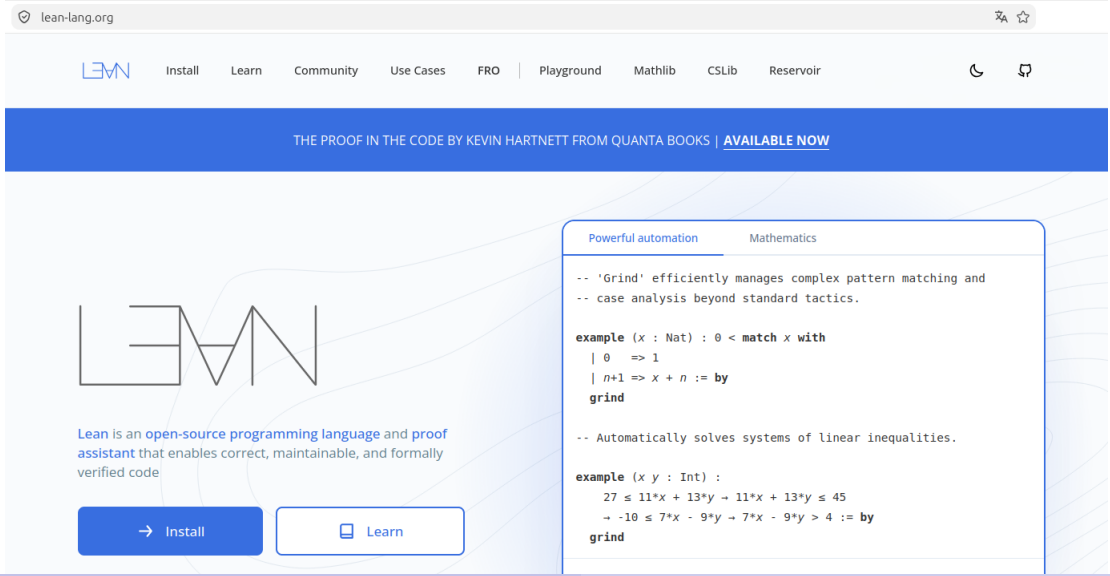
Disponível no Lean Web

O ambiente `live.lean-lang.org` inclui Mathlib — basta importar.

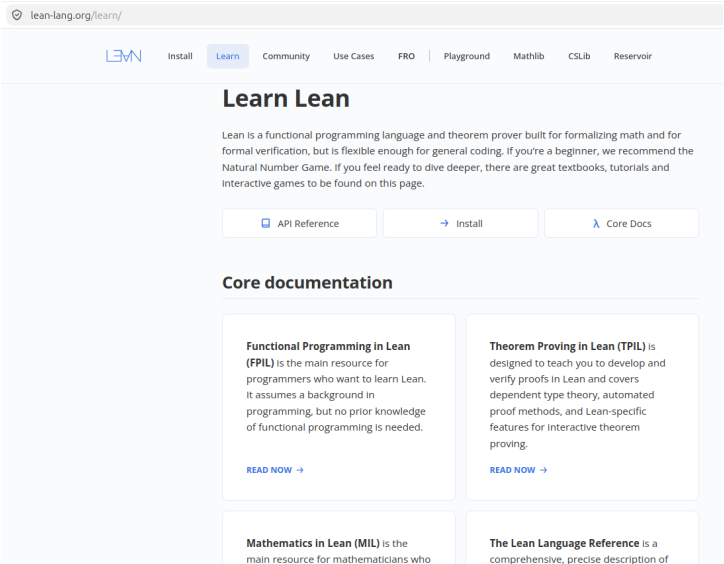
O que é o Mathlib?



Onde achar o Lean?



Onde achar o Lean?



Resumo do Dia 1

1. **Motivação:** provas matemáticas podem conter erros; um assistente de prova confere mecanicamente *cada passo* lógico.
2. **IA e Lean:** os LLMs trabalham com probabilidades e podem alucinar; o Lean dá um veredito binário: a prova é aceita ou recusada.
3. **O que o Lean não faz:** não testa casos nem decide sozinho se um enunciado é verdadeiro. Isso é o que fazem os provadores SAT/SMT.
4. **O que o Lean faz:** trabalha com teoria de tipos dependentes. Pela correspondência de Curry–Howard, proposições são tipos e provas são termos; o *kernel* confere se `t : P`.
5. **Pesquisa:** IA e Lean já estão sendo usados em problemas de pesquisa em matemática, e o kernel é o que torna esses resultados confiáveis.
6. **Prática:** `live.lean-lang.org`, `#eval`, `def`, `theorem`, termos $\times$ táticas e o estado da prova (Infoview).
7. **Ecossistema:** a Mathlib e onde continuar estudando.

Resumo do Dia 1

A ideia central

Provar é construir um objeto do tipo certo. O Lean confere; você guia.

Amanhã

Axiomas de Peano no Lean + Números Naturais → escrever provas juntos.